AI-Assisted Phishing URL Risk Checker

Rule-Based URL Analysis with Optional Amazon Bedrock Explanations





Project Type	**Personal Hands-on AI and Cloud Cybersecurity Project**
Project Domain	AI-Assisted Phishing URL Risk Checker
Cloud Platform	Amazon Web Services (AWS)
AWS Region	Asia Pacific (Singapore) — ap-southeast-1
Bedrock base model	amazon.nova-micro-v1:0
Technologies	Python, Streamlit, Boto3 + AWS CRT, Amazon Bedrock, Amazon Nova Micro, APAC inference profile, AWS IAM / AWS CLI v2, AWS Budgets

Prepared By : Chanmith Jayathilake

Role : Cloud & Security Engineer (Personal Project)

Project Status : Completed

Documentation Date : November 2026

Copyright Notice

© 2026 P.C.L.D Jayathilake. All rights reserved.

This documentation is the original intellectual property of the author. It is protected under applicable national and international copyright laws.

No part of this document may be copied, reproduced, distributed, published, transmitted, modified, translated, or stored in any form or by any means-electronic, mechanical, photocopying, recording, or otherwise-without the prior written permission of the author, except for brief quotations used for educational or non-commercial purposes with proper attribution.

Unauthorized use, duplication, or distribution of this material without permission may result in legal action.

The author retains full moral and intellectual property rights over this work.

Ethical Declaration of Original Work and Authorization

I hereby formally declare that this documentation is an original work created by me and represents my independent effort, knowledge, and practical understanding of the subject matter. All concepts, configurations, explanations, diagrams, workflows, and technical descriptions included in this document have been developed based on my academic studies, professional training, hands-on practice, and personal analysis.

I confirm that this work has not been copied, duplicated, or plagiarized from any other source. Where external materials, standards, documentation, tools, or references have been consulted for learning or verification purposes, they have been appropriately acknowledged and credited in a fair and ethical manner. No part of this document intentionally infringes upon the intellectual property rights of any individual, organization, or institution.

This documentation has been prepared in accordance with ethical, academic, and professional standards. It is intended solely for educational, informational, and professional knowledge-sharing purposes. Any similarities to existing works are purely coincidental or arise from commonly accepted technical practices and industry-standard terminology.

Authorization Statement

I, the undersigned author, retain full ownership and intellectual property rights over this documentation. I authorize the public viewing and reading of this work through approved platforms where it is published or shared by me.

Unless explicitly stated otherwise, no individual, organization, or third party is permitted to copy, reproduce, modify, translate, redistribute, sell, or use this material for commercial purposes without my prior written consent.

Limited use of this documentation is authorized for educational and non-commercial purposes only, provided that:

- Proper credit is clearly given to the author
- The content is not altered in a way that misrepresents the original work
- The use does not violate ethical, legal, or professional standards

Any unauthorized reproduction, distribution, or misuse of this material may result in legal or ethical consequences in accordance with applicable intellectual property laws.

I accept full responsibility for the accuracy, integrity, and originality of the content presented in this documentation.

Field: Network Engineering & Ethical Hacking
Date: November 2026

TABLE OF CONTENTS

1. Introduction

2. Problem Domain

3. Proposed Solution

4. Project Aim

5. Objectives

6. System Architecture with AWS

    6.1 File Structure

7. Implementation Technologies

8. Setup and Configuration

    PART A | LOCAL SETUP

    1. Run the Application on Windows

        1.1 Open the terminal in VS Code and Check Python is installed

        1.2 Giving executable permission and activating the dedicated virtual environment

        1.3 Install the dependencies

        1.4 Start the web application

    2. Test the scanner

        2.1 Test: https://www.python.org

        2.2 Test: http://localhost:8501

        2.3 Test: trusted.example@attacker.example/login

        2.4 Test: http://192.0.2.1/login

        2.5 Test: https://login-verify.example.net

PART B | AWS Bedrock Integration with CLI

3. AWS Budget and IAM User Creation

3.1 Select Asia Pacific (Singapore) and create an AWS Budget

3.2 Create the IAM user with the login policy (phishing-bedrock-demo-user)

3.3 Create a separate Bedrock policy and attach it

3.4 Created Bedrock policy

3.5 Attaching the Bedrock policy to User: phishing-bedrock-demo-user

4. Find Nova Micro in Singapore Bedrock

4.1 Open the Bedrock Console and Model Catalog

4.2 Confirm the profile identifier

5. Authenticate VS Code as your IAM user

5.1 Install AWS CLI v2 and check Python SDK

5.2 Run browser-based CLI login

5.3 Verify the actual identity

6. Verify APAC Nova Micro with the CLI

6.1 Set the AWS CLI profile Region once

6.2 Inspect the APAC Nova Micro profile

7. Run a standalone Bedrock smoke test

7.1 Create and run bedrock_smoke_test.py

8. Run your app with Bedrock

8.1 Activate, set variables and verify Python

8.2 Launch Streamlit and test with AI (Bedrock)

8.3 Phishing URL Risk Analysis: Standard Detection vs AI-Enhance Explanation

9. Conclusion

10. References

1. INTRODUCTION

Phishing is a social-engineering technique in which an attacker uses misleading messages or web addresses to trick users into disclosing information or visiting malicious services.

This project provides a small educational tool that highlights suspicious URL structures and explains the reasons behind each result. The application is designed to run locally in VS Code, with optional AI explanations hosted by Amazon Bedrock.

	PROJECT PHASE	SCOPE
	PART A: Local Python + Streamlit	Offline URL parsing, structural security rules, scores and a browser interface. No AWS services required.
	PART B: Optional AWS Bedrock	Secure AWS CLI identity; Amazon Nova Micro through a supported inference profile; AI text explaining existing findings.

The system is a proof of concept. Its score is not a machine-learning probability or evidence of website safety; it never visits the submitted URL.

2. PROBLEM DOMAIN

People frequently encounter unfamiliar links through email, chat and websites. The actual destination can be obscured through user-information syntax, IP-address hosts, unusually complex domains or persuasive words in a path.

Non-technical users may miss these warning signs. Traditional link-risk solutions also require datasets, network access or paid reputation services that can be excessive for a small demonstration.

3. PROPOSED SOLUTION

A Streamlit interface accepts a URL and passes it to a local Python module. The module checks observable structural indicators such as HTTP, IP-based hosts, embedded @ user information, Punycode, unusually long hostnames, subdomain depth, nonstandard ports, URL shorteners and selected attention-related words.

It then calculates a transparent heuristic score and displays Low, Medium or High indicators. With explicit user selection, Amazon Bedrock generates a brief explanation of those findings via Boto3.

4. PROJECT AIM

To design and demonstrate a low-complexity, explainable phishing URL risk checker that combines fundamental cybersecurity checks with optional AWS-based generative AI while following basic identity, privacy and cost-control practices.

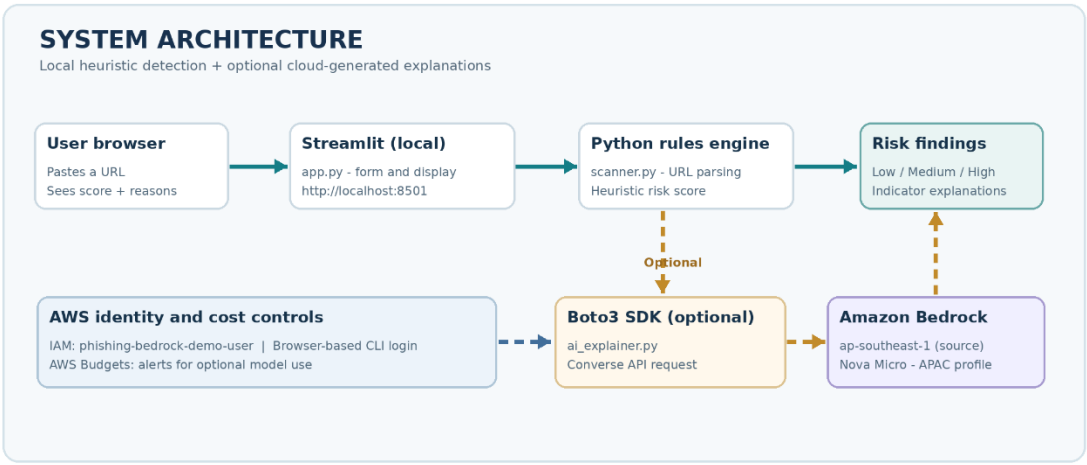
5. OBJECTIVES

- Develop an easy-to-use Streamlit interface for URL input and display.
- Implement offline URL-structure checks with readable reasons and a bounded 0–100 heuristic score.
- Demonstrate Low, Medium and High labels using synthetic and familiar test URLs.
- Integrate the Amazon Bedrock Converse API as an optional explanation layer.
- Secure AWS access using a dedicated IAM user, constrained permissions and temporary CLI login.
- Validate scanner rules with unit tests and verify the Bedrock connection with a standalone smoke test.
- Document deployment steps, AWS expenditure controls and known detection limitations.

6. SYSTEM ARCHITECTURE WITH AWS

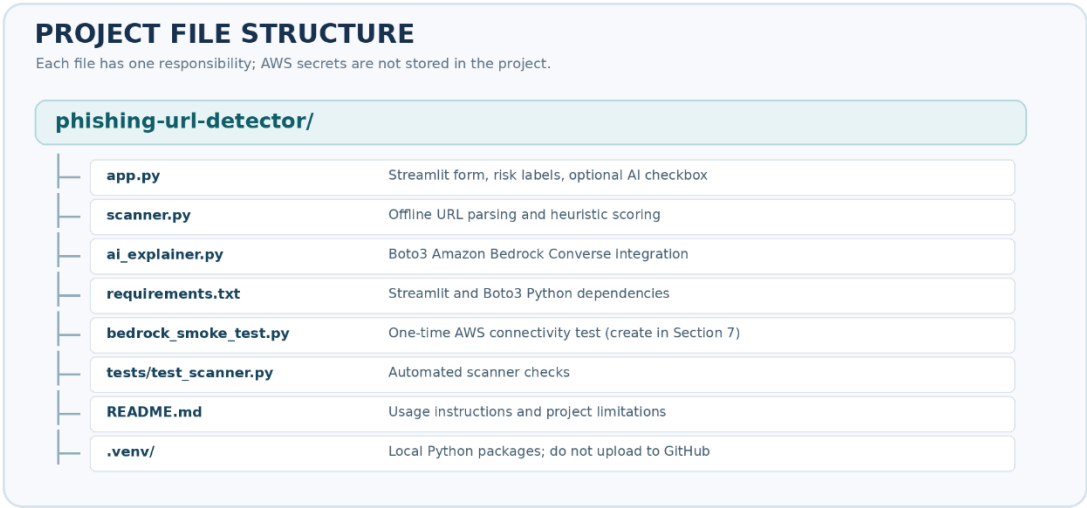
The diagram shows the two-tier design:

local URL inspection runs independently of AWS, while AI explanations are requested only when the user enables them. The CLI identity authorises Boto3; AWS Budgets is a governance service, not part of the inference data path.



6.1 File Structure

The project uses separate modules for the user interface, URL analysis, AI integration, tests and dependencies. The smoke-test file is created during the AWS configuration stage and is not part of the original ZIP.



7. IMPLEMENTATION TECHNOLOGIES

TECHNOLOGY	PURPOSE
Python 3.10+	Application logic and URL processing.
Streamlit	Browser UI for local demonstration.
urllib.parse / ipaddress	Built-in URL parsing and IP-address checks.
Boto3 + AWS CRT	AWS SDK and browser-based session credential support.
Amazon Bedrock	AI-generated explanation of existing risk findings.
Amazon Nova Micro	Low-cost text model; base model amazon.nova-micro-v1:0.
APAC inference profile	Runtime modelId for supported geographic cross-Region requests.
AWS IAM / AWS CLI v2	Restricted identity, browser login, regional CLI control.
AWS Budgets	Alerts for unexpected spending; not a billing shut-off.
VS Code / PowerShell	Windows development and commands.

8. SETUP AND CONFIGURATION

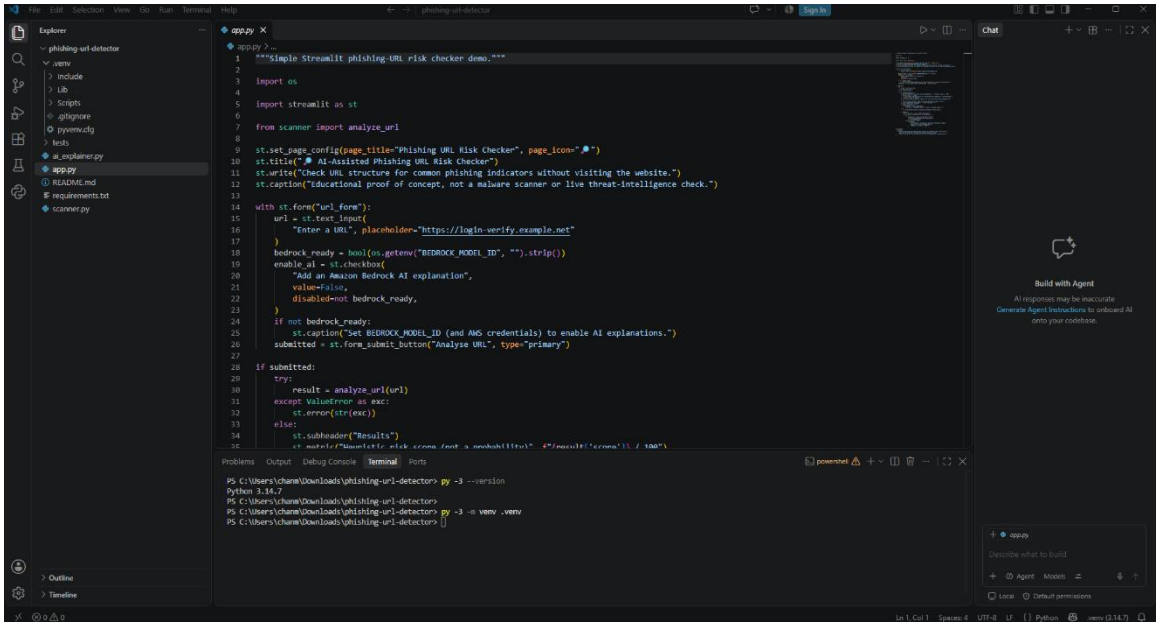
The instructions below follow the project implementation sequence. Screenshot locations are intentionally labelled: no screenshots from the user's AWS account or VS Code session were supplied. Replace these placeholders with real screenshots before formal submission, and mask credentials, account IDs and other sensitive information.

PART A | LOCAL SETUP

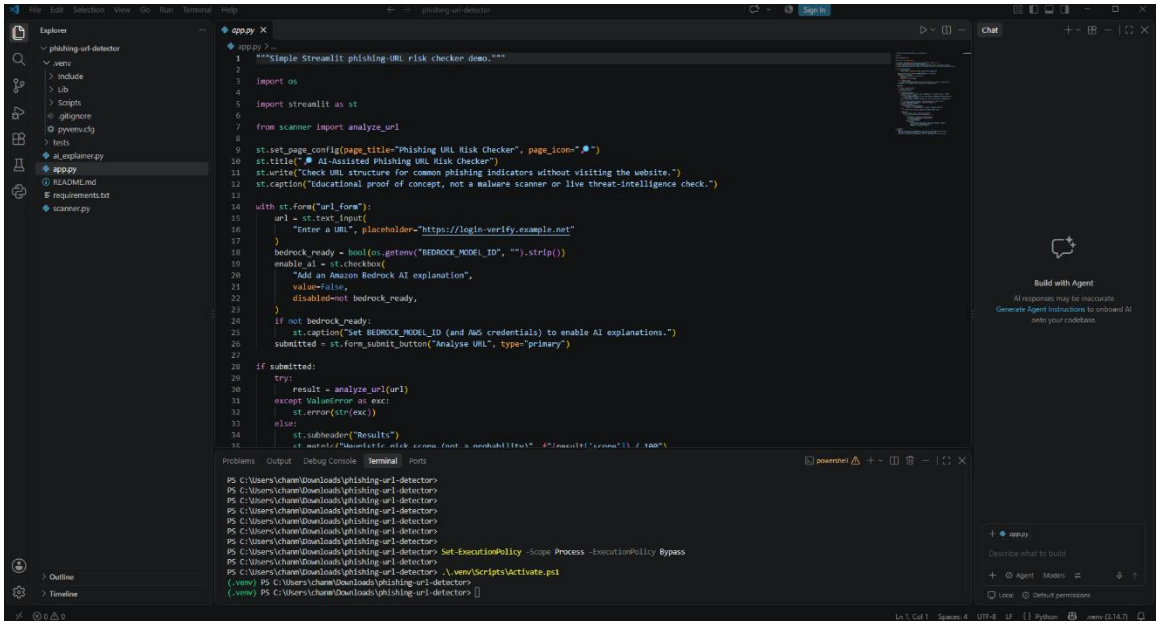
1. Run the Application on Windows

This section prepares the existing starter project on a Windows computer, installs its Python dependencies and starts the Streamlit interface. All URL checks work locally before connecting to AWS.

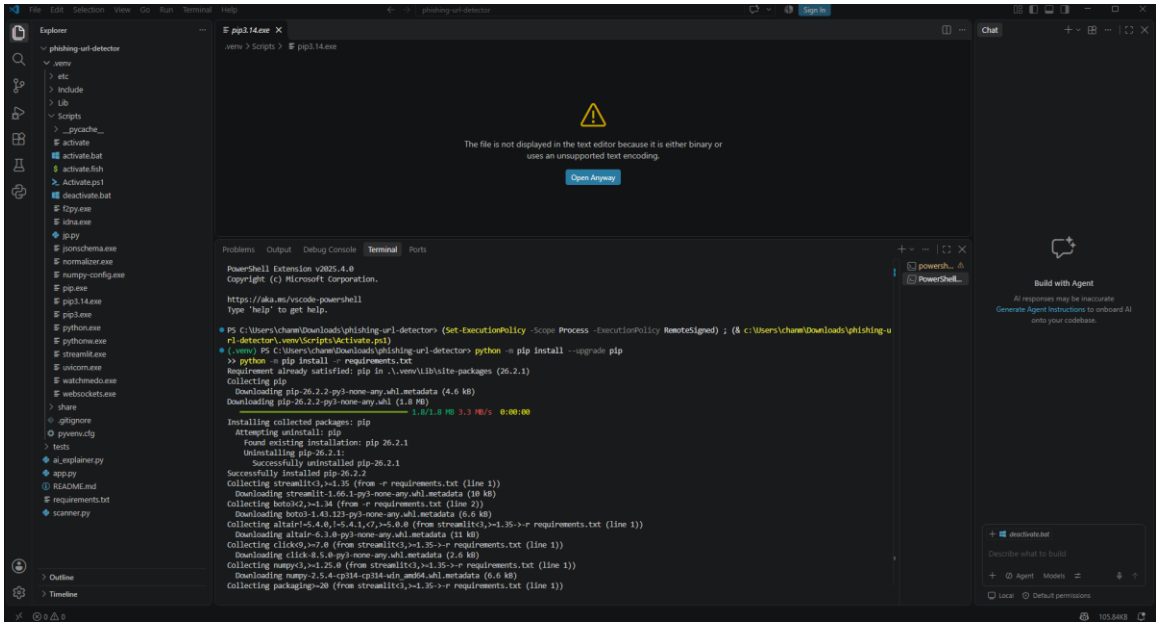
1.1 Open the terminal in VS Code and Check Python is installed



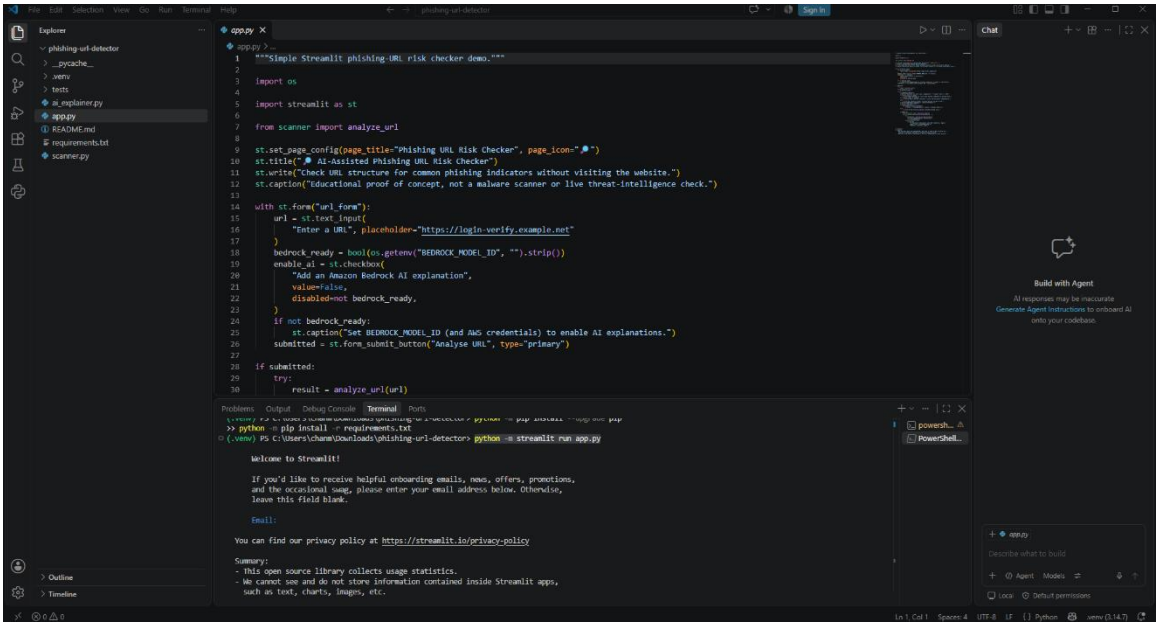
1.2 Giving executable permission and activating the dedicated virtual environment



1.3 Install the dependencies



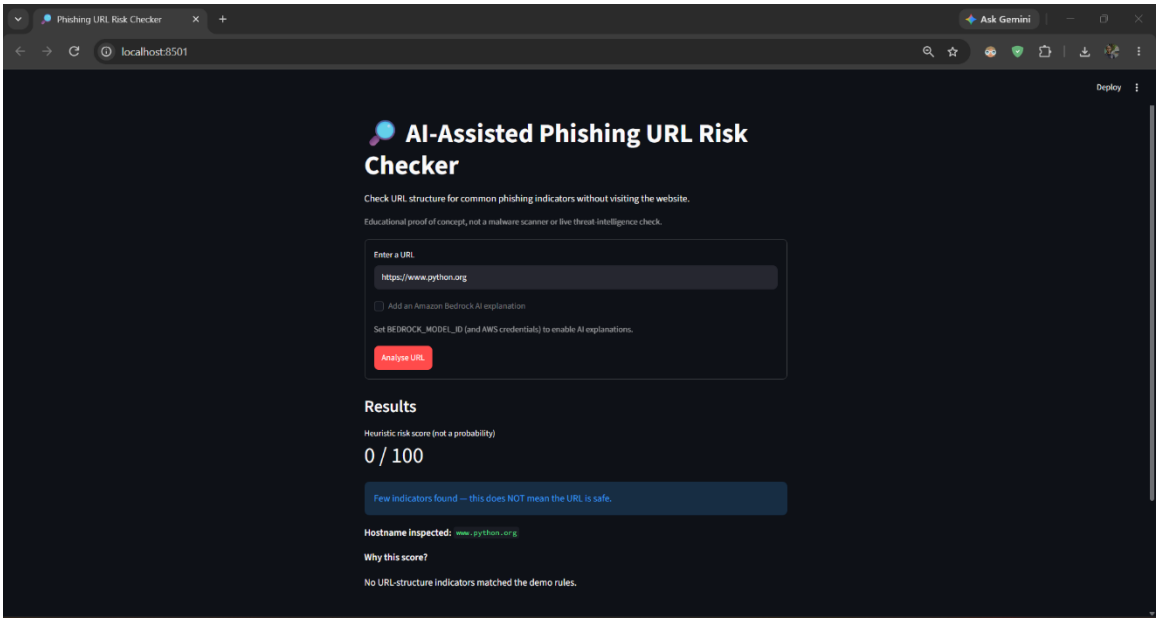
1.4 Start the Web Application



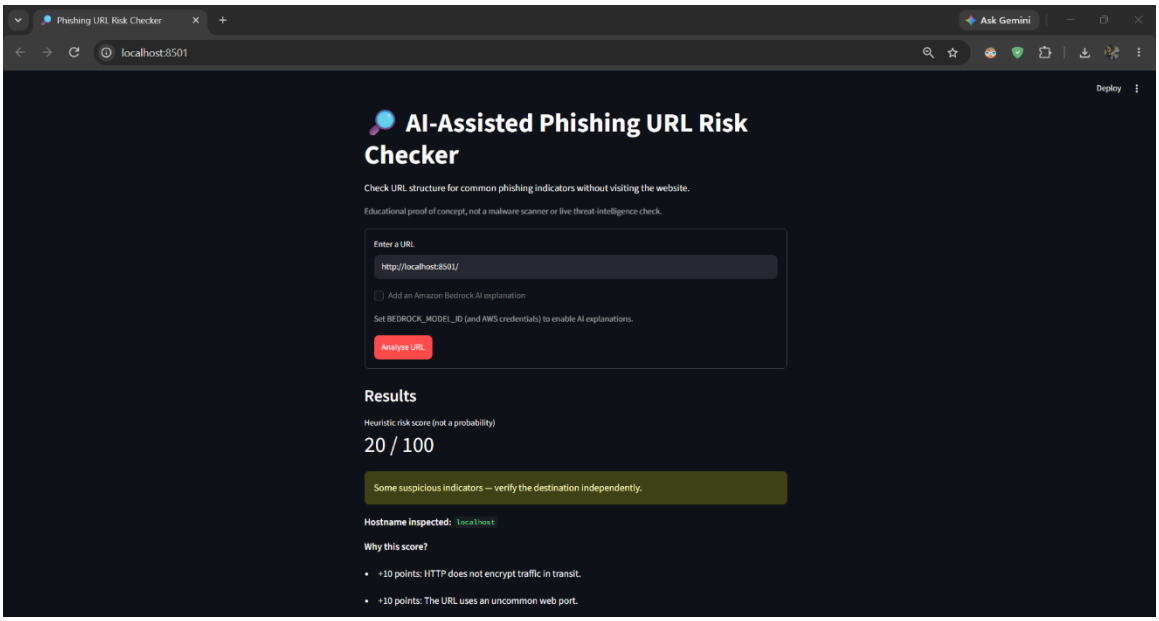
2. Test the Scanner

The scanner checks URL structure without opening a target website. These test cases intentionally demonstrate different indicator combinations; the Low/Medium/High labels are rule-based and should not be treated as confirmed safe or malicious classifications.

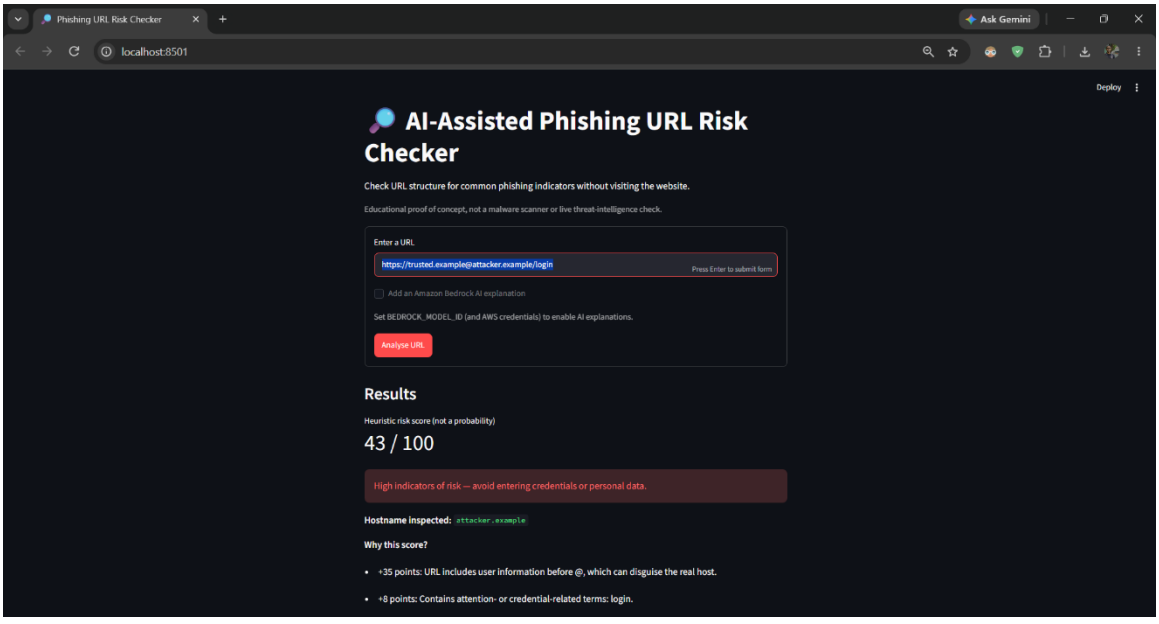
2.1 Test: https://www.python.org



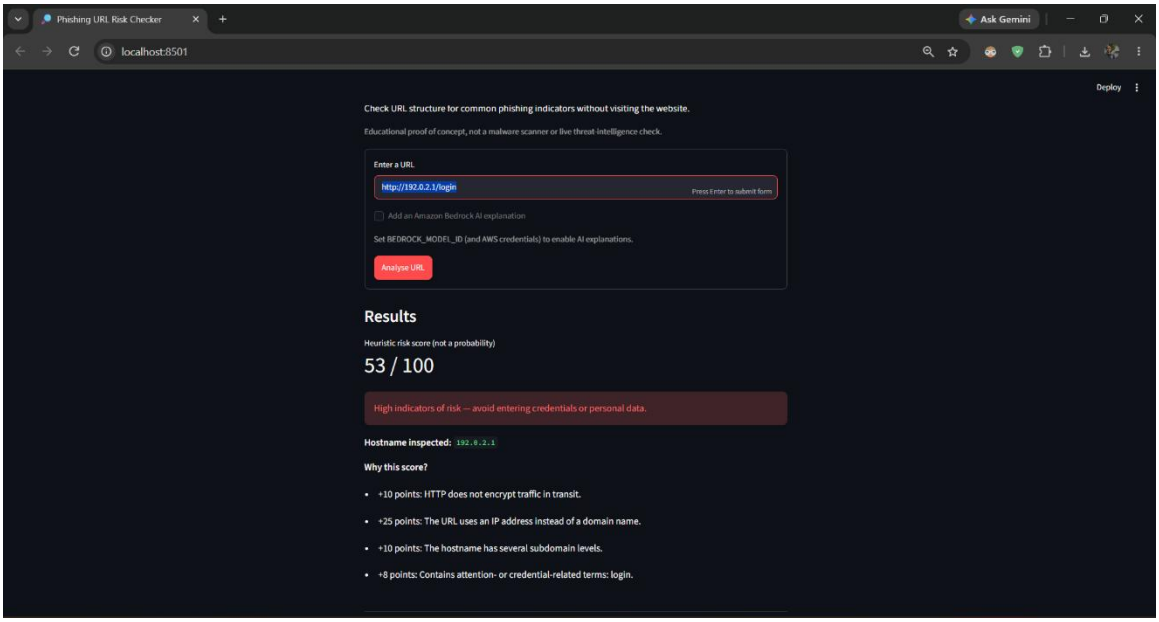
2.2 Test: http://localhost:8501



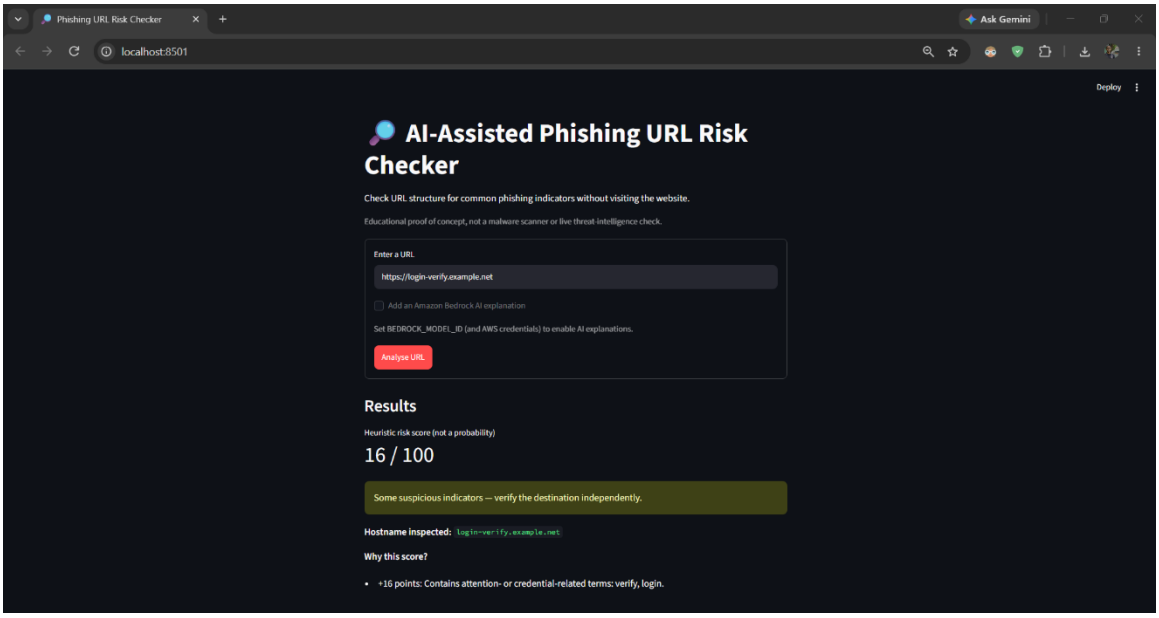
2.3 Test: trusted.example@attacker.example/login



2.4 Test: http://192.0.2.1/login



2.5 Test: https://login-verify.example.net

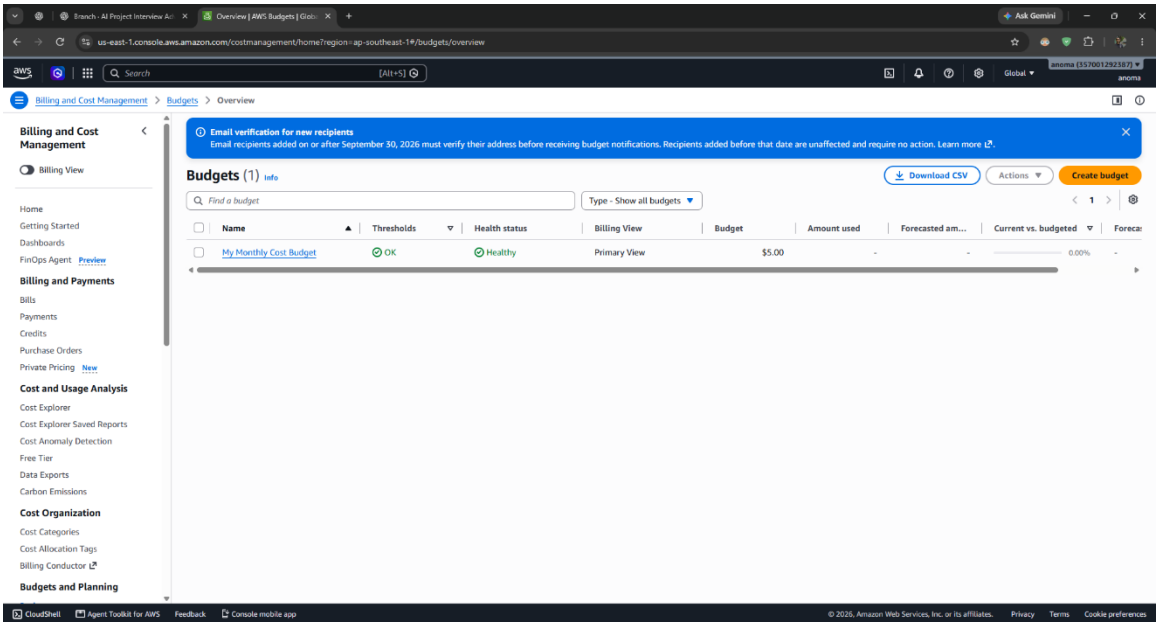


PART B | AWS Bedrock Integration with CLI

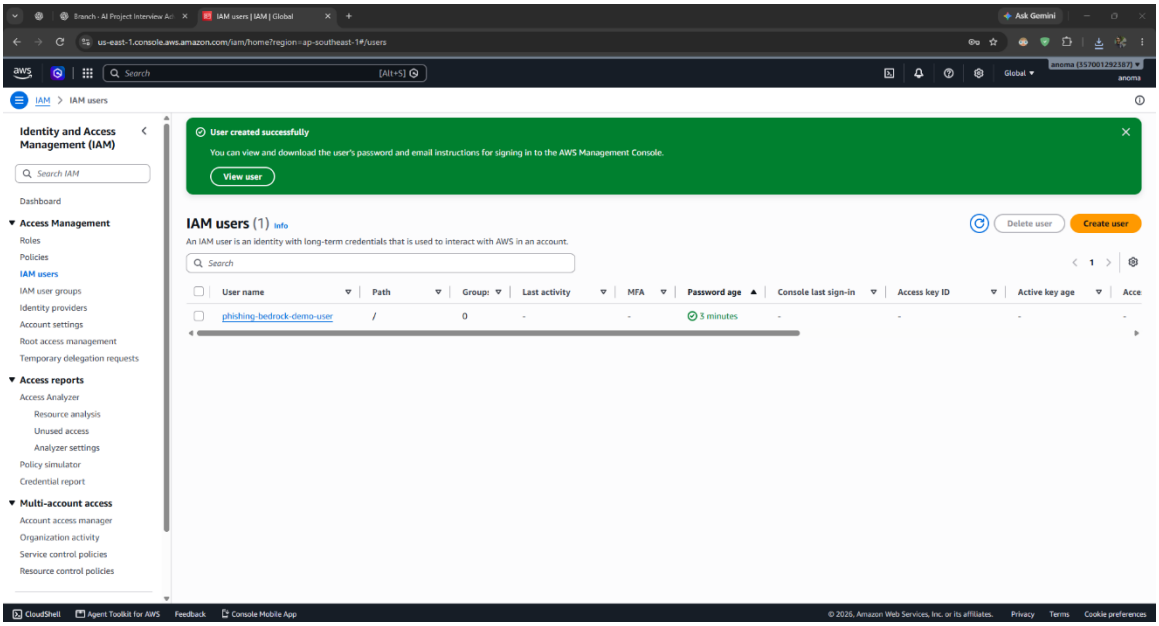
3. AWS Budget and IAM User Creation

Configure the personal AWS account for controlled, low-volume AI testing. A separate IAM user receives only the permissions needed to sign in and invoke the intended Bedrock inference profile.

3.1 Select Asia Pacific (Singapore) and create an AWS Budget

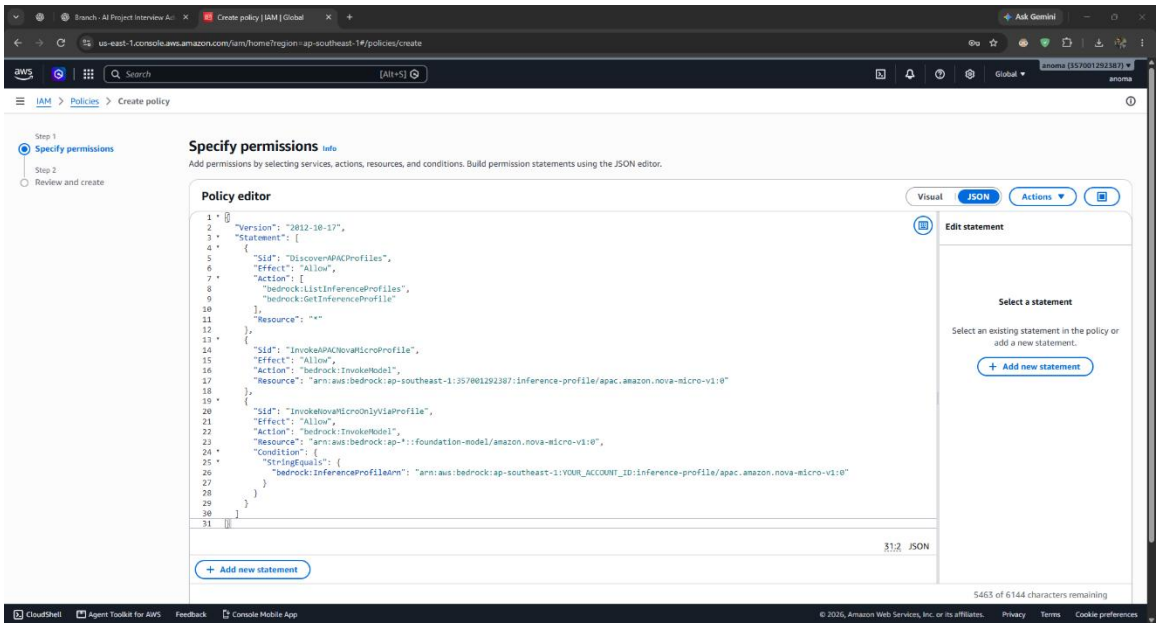


3.2 Create the IAM user with the login policy (phishing-bedrock-demo-user)

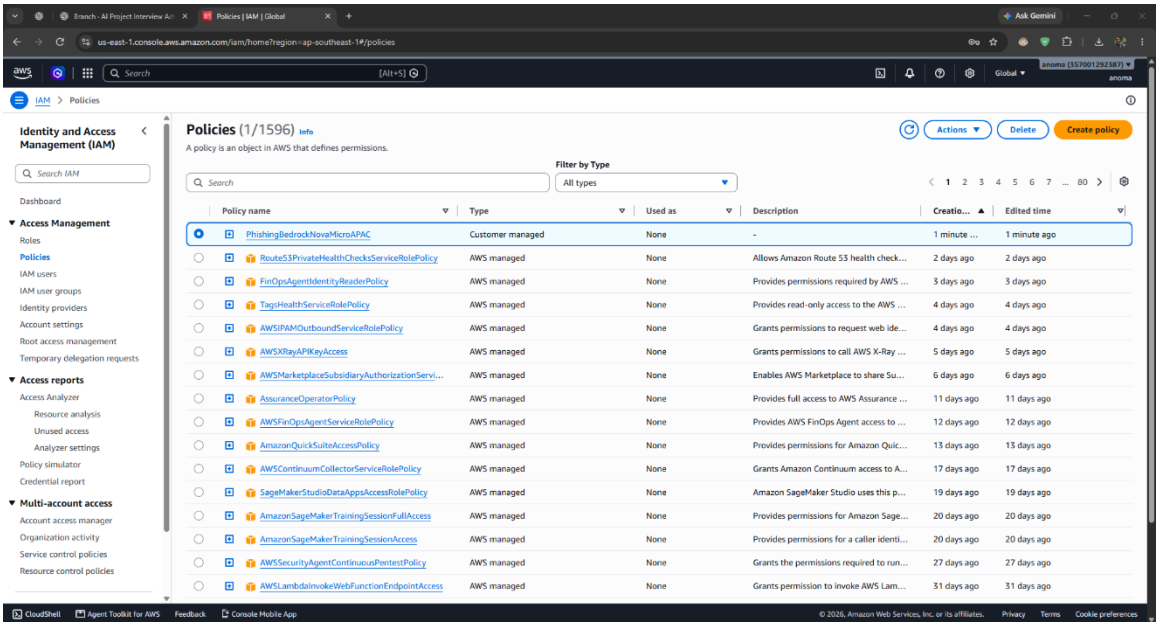


User : phishing-bedrock-demo-user
AWS Managed Policy : SignInLocalDevelopmentAccess

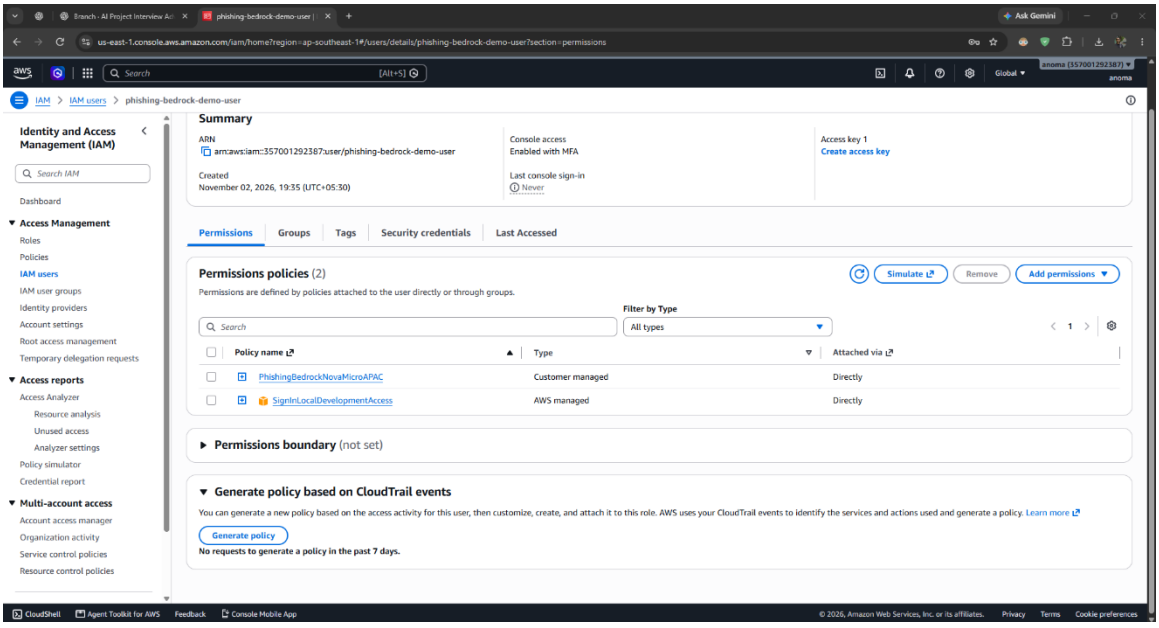
3.3 Create a separate Bedrock policy and attach it



3.4 Created Bedrock policy



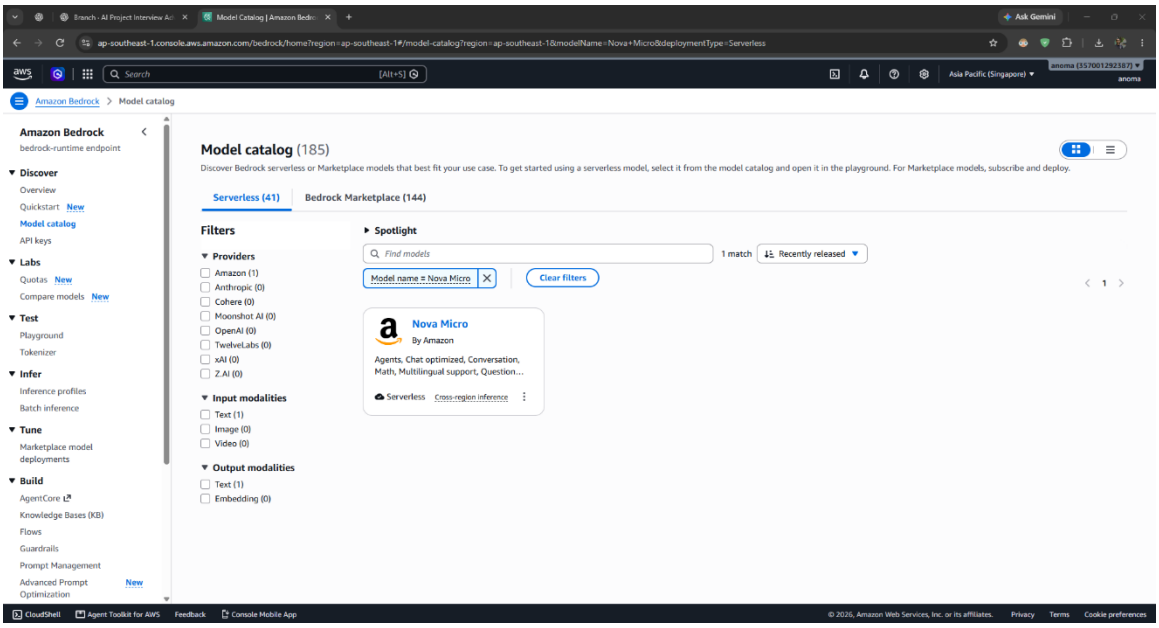
3.5 Attaching the Bedrock policy to User: phishing-bedrock-demo-user

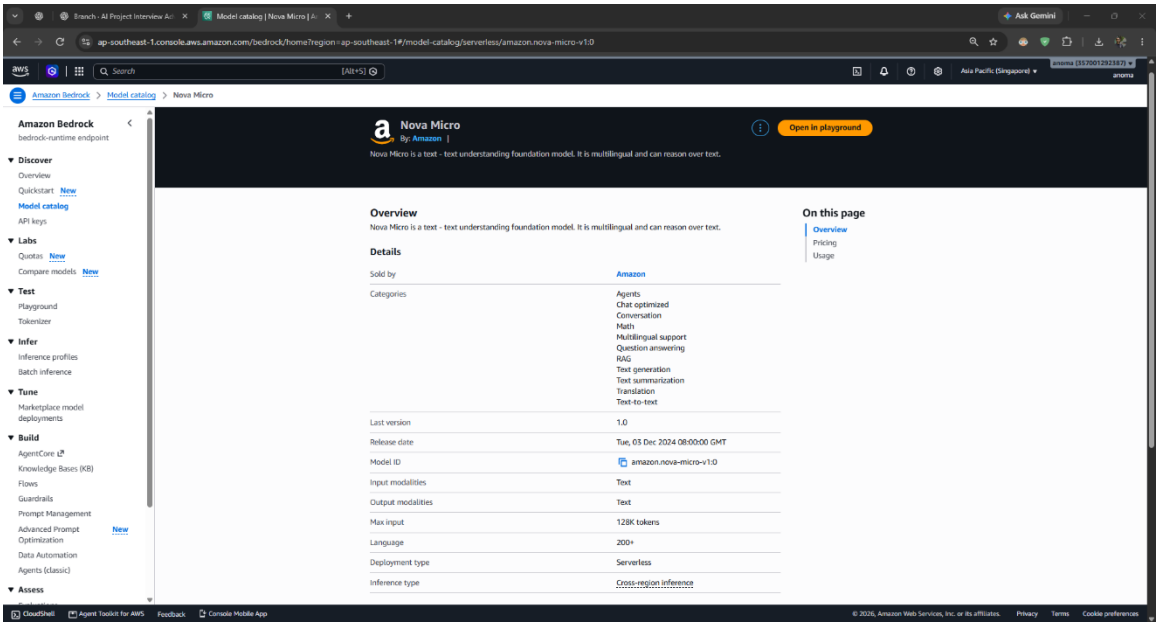


4. Find Nova Micro in Singapore Bedrock

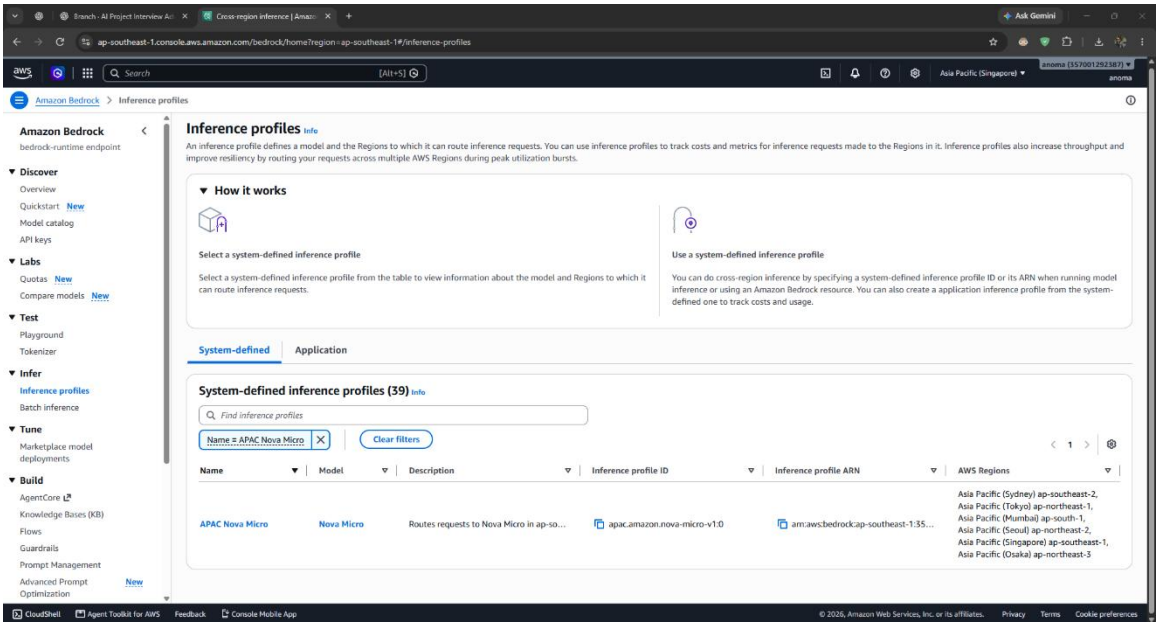
Amazon Nova Micro is the selected text model. In Singapore, the application must use an available geographic inference profile rather than assuming the base model ID is directly invokable in-Region.

4.1 Open the Bedrock Console and Model Catalog





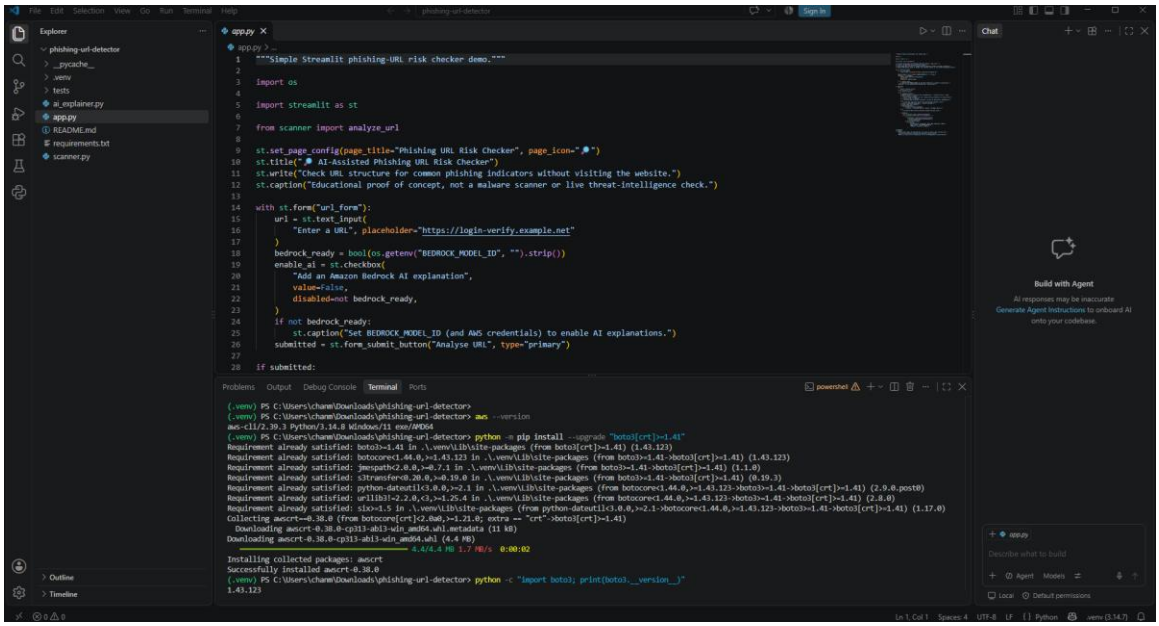
4.2 Confirm the profile identifier



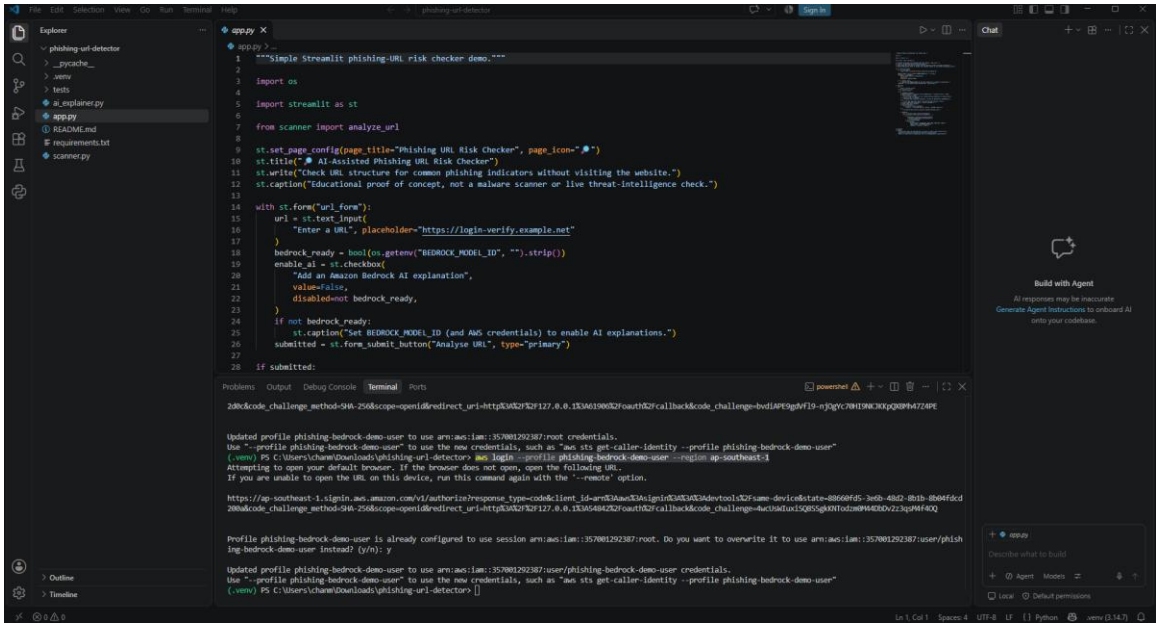
5. Authenticate VS Code as your IAM user

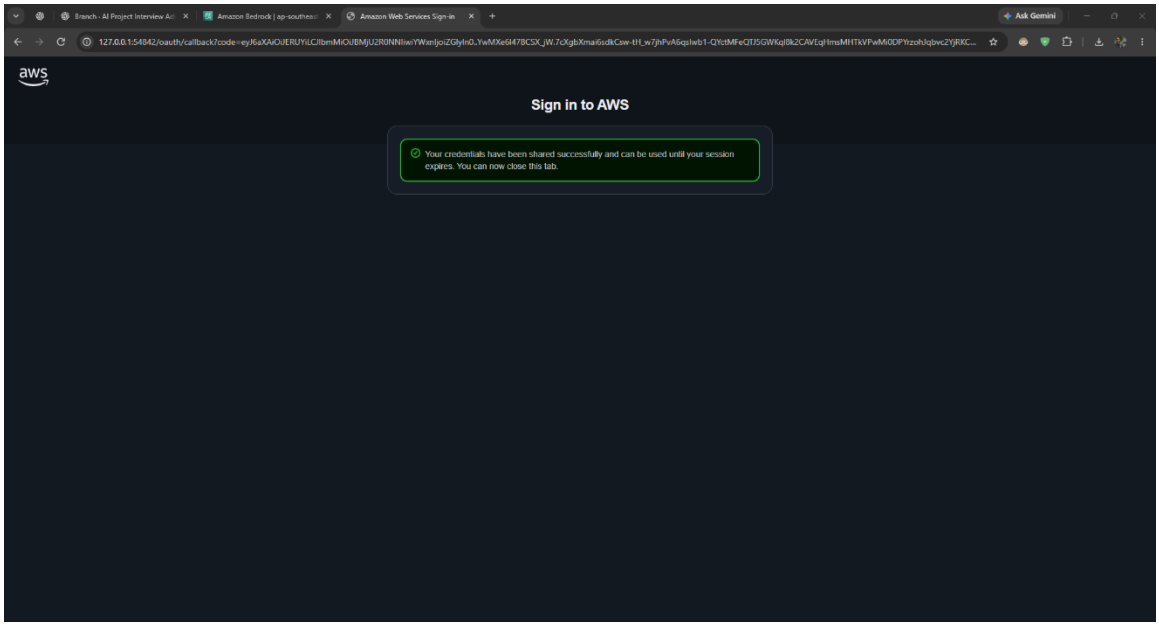
A browser-based AWS CLI login provides short-lived credentials so Python can use the dedicated IAM identity without storing permanent access keys in the project.

5.1 Install AWS CLI v2 and check Python SDK

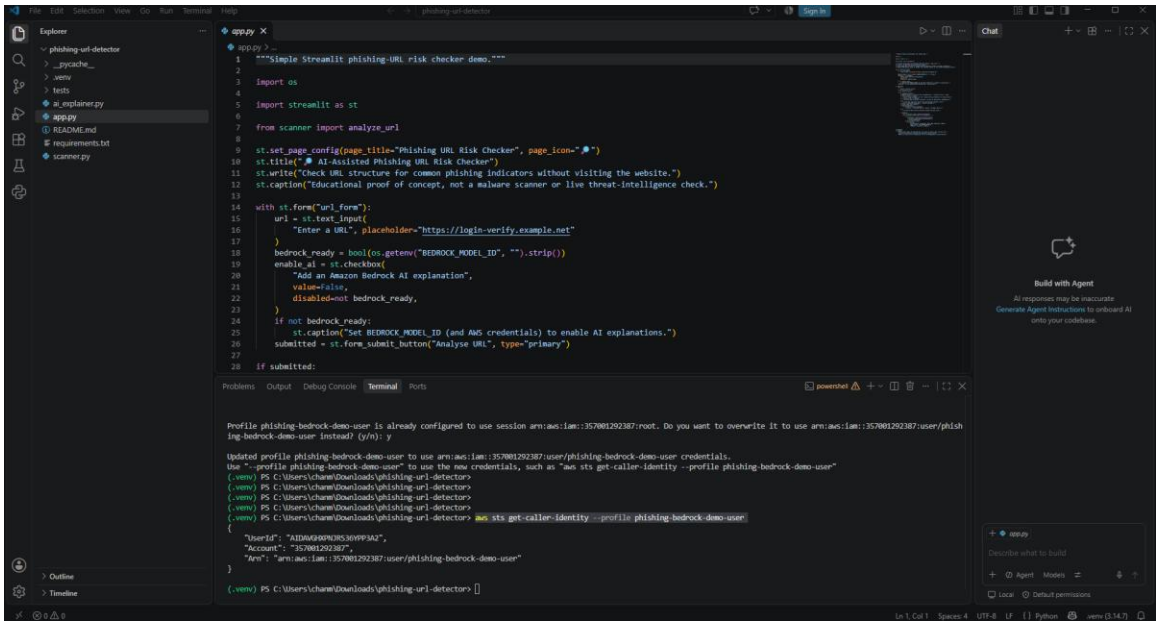


5.2 Run browser-based CLI login





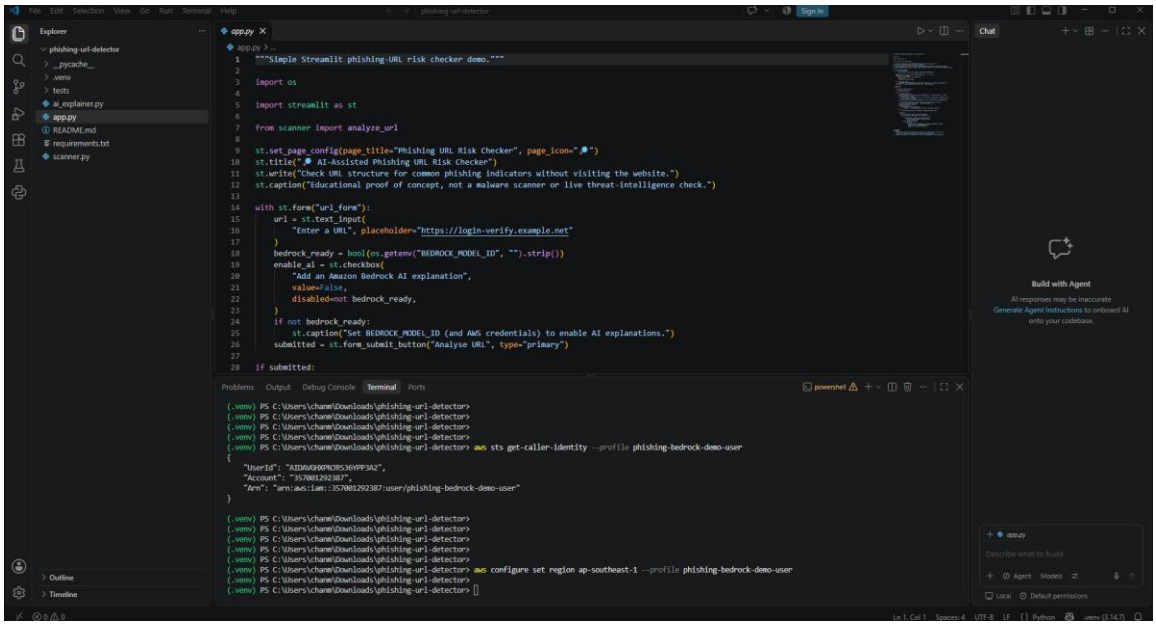
5.3 Verify the actual identity



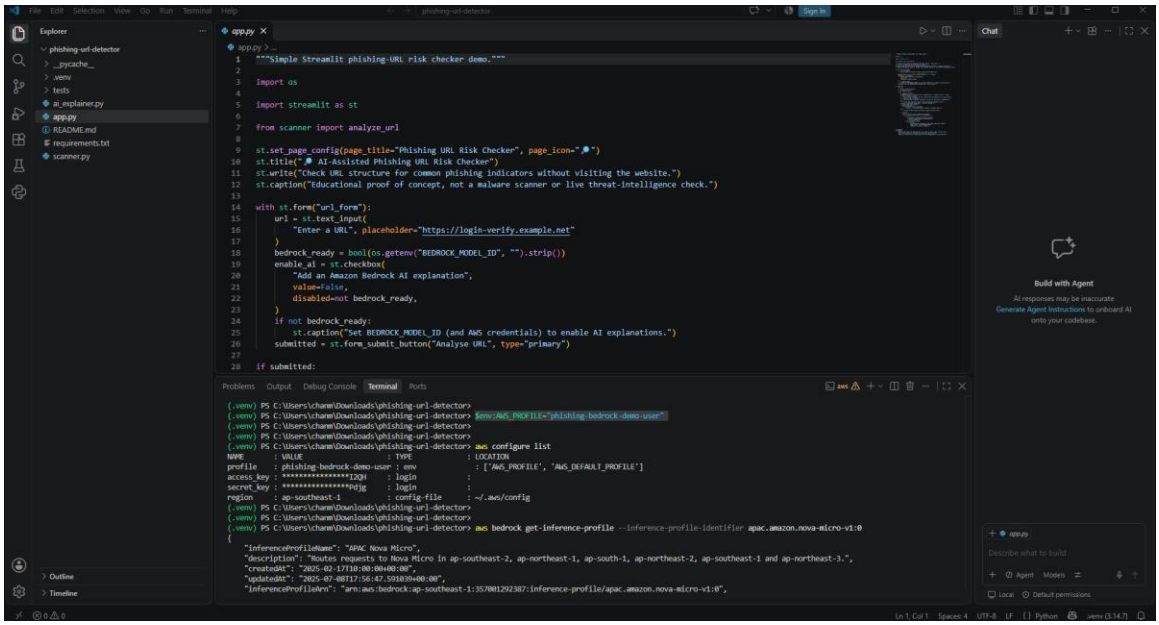
6. Verify APAC Nova Micro with the CLI

Use the Bedrock control-plane commands to verify the configured source Region and the exact inference profile that your IAM identity can access.

6.1 Set the AWS CLI profile Region once



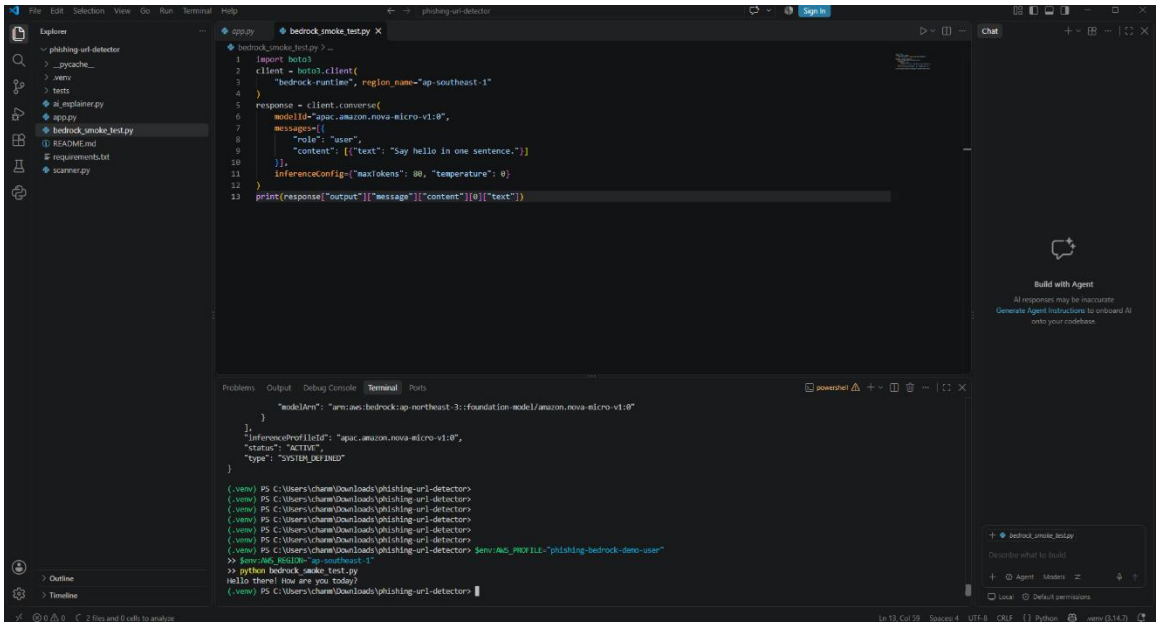
6.2 Inspect the APAC Nova Micro profile



7. Run a standalone Bedrock smoke test

A short Python call verifies the entire path - identity, Region, model profile and the Converse API - before the AI option is enabled in Streamlit.

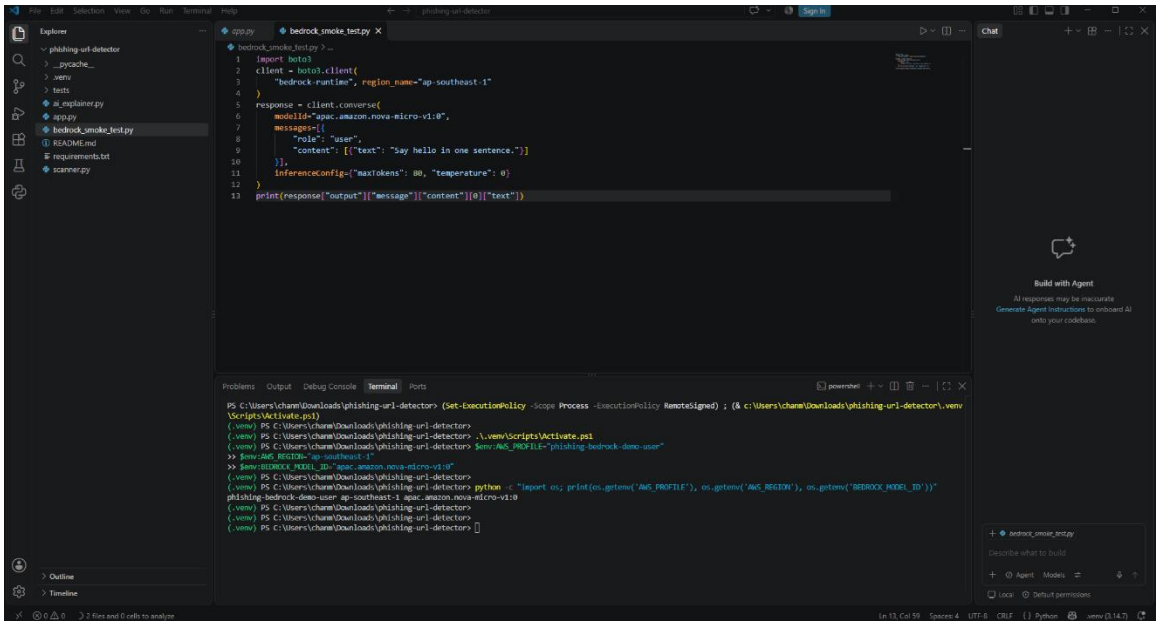
7.1 Create and run bedrock_smoke_test.py



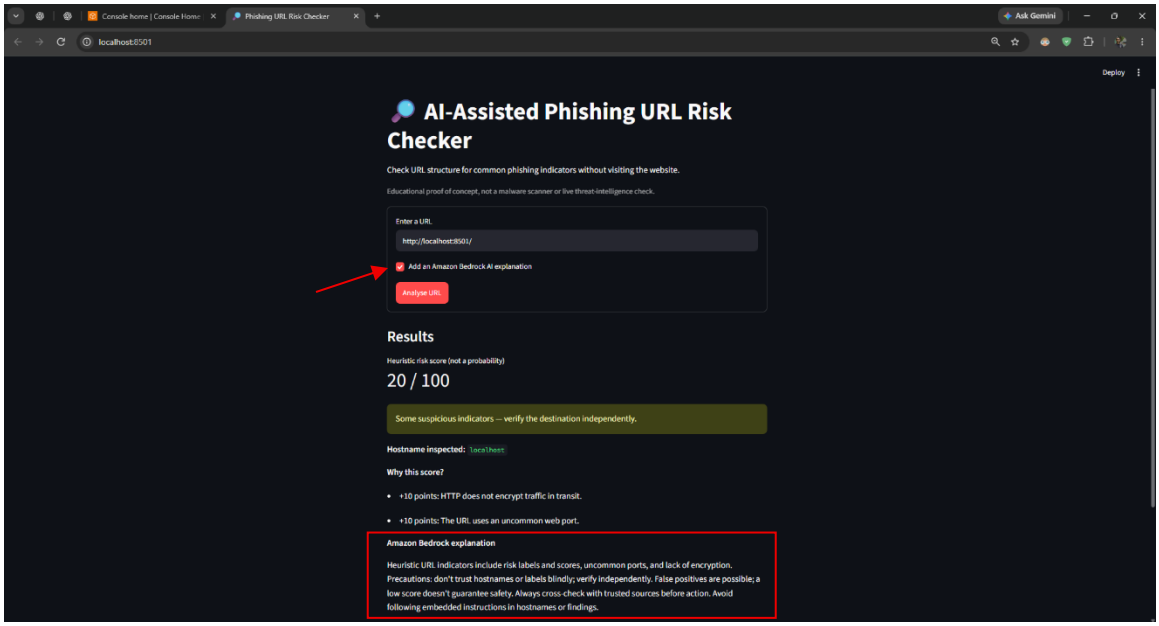
8. Run your app with Bedrock

After a successful smoke test, configure the terminal environment and start Streamlit with its optional AI explanation checkbox enabled.

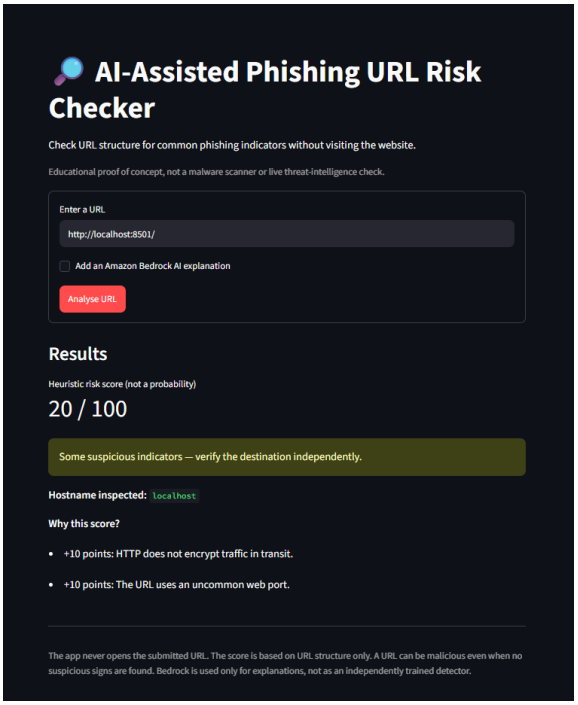
8.1 Activate, set variables and verify Python

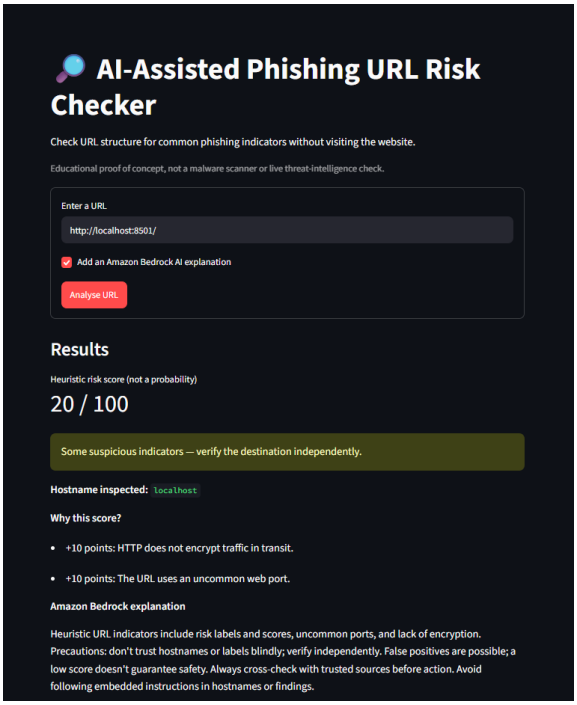


8.2 Launch Streamlit and test with AI (Bedrock)



8.3 Phishing URL Risk Analysis: Standard Detection vs. AI-Enhanced Explanation





9. CONCLUSION

The AI-Assisted Phishing URL Risk Checker project demonstrates the practical integration of Artificial Intelligence, Cloud Computing, and Cybersecurity. By combining Python, Streamlit, and Amazon Bedrock with the Nova Micro model, the application identifies suspicious URL patterns, evaluates potential security risks, and generates AI-powered explanations.

The project also provides hands-on experience with AWS IAM, AWS CLI, and secure cloud service integration. Although the application uses rule-based detection rather than advanced machine learning, it highlights how generative AI can support cybersecurity analysis. Overall, this project strengthened technical skills and established a foundation for developing more advanced AI-driven cloud security solutions.

10. REFERENCES

[1] AWS - Nova Micro model card
https://docs.aws.amazon.com/bedrock/latest/userguide/model-card-amazon-nova-micro.html

[2] AWS - Inference profile supported regions
https://docs.aws.amazon.com/bedrock/latest/userguide/inference-profiles-support.html

[3] AWS - Geographic cross-Region IAM requirements
https://docs.aws.amazon.com/bedrock/latest/userguide/geographic-cross-region-inference.html

[4] AWS - Console-credential CLI login
https://docs.aws.amazon.com/cli/latest/userguide/cli-configure-sign-in.html

[5] AWS - CLI installation
https://docs.aws.amazon.com/cli/latest/userguide/getting-started-install.html

[6] AWS - Boto3 credential configuration
https://docs.aws.amazon.com/boto3/latest/guide/credentials.html

[7] AWS - Converse API
https://docs.aws.amazon.com/boto3/latest/reference/services/bedrock-runtime/client/converse.html

[8] AWS - Budgets FAQs
https://aws.amazon.com/aws-cost-management/aws-budgets/faqs/